STUDYOS

Personal Learning, Productivity & Competitive Examination Tracking System

Complete Project Documentation
React.js + Vite • Python Flask • SQLite • SQLAlchemy • REST API

Core principle: the complete application works without paid AI APIs such as Gemini, ChatGPT, Claude, or other paid external AI services.

Table of Contents

1. Project Overview

2. Problem Statement

3. Project Objectives

4. Scope of the System

5. Core Features

6. Technology Stack

7. System Architecture

8. Module Architecture

9. User Workflow

10. Dashboard Design

11. Course & Subject Management

12. Competitive Exam Management

13. Syllabus Tracking

14. Study Session & Timer

15. Reading Tracker

16. Goals & Task Management

17. Calendar & Planner

18. PYQ & Question Bank

19. Mock Test Analytics

20. Revision Engine

21. Mistake Book

22. Notes & Knowledge Library

23. Focus & Distraction Tracking

24. Analytics Engine

25. Smart Recommendation Engine Without Paid AI

26. Study DNA & Personal Insights

27. Reports & Export

28. Database Design

29. REST API Design

30. React Frontend Structure

31. Flask Backend Structure

32. Security

33. Offline-First Strategy

34. Development Roadmap

35. Testing Strategy

36. Deployment

37. Future Enhancements

38. Project Novelty

39. Suggested Final-Year Project Description

40. Conclusion

1. Project Overview

StudyOS is a personal learning and competitive-examination management system designed for day-to-day use. It combines study-time tracking, course management, competitive-exam preparation, syllabus progress, reading activity, goals, tasks, revision, practice questions, mock tests, mistakes, productivity analytics, and personalized recommendations in one application.

The system is designed primarily for a student who wants to understand not only how much time was spent studying, but also what was studied, when it was studied, how consistently it was studied, which topics are weak, which revisions are overdue, and how preparation is progressing.

The project intentionally avoids dependency on paid AI APIs. Core recommendations are generated using deterministic rules, statistics, scheduling algorithms, performance analysis, and locally stored user data. Optional local AI can be added later without changing the core architecture.

2. Problem Statement

Students commonly use separate applications for timers, calendars, notes, task lists, reading, mock tests, and exam preparation. This fragmentation makes it difficult to obtain a complete picture of learning behavior.

- Study time is often recorded without linking it to a specific course, subject, and topic.
- A completed syllabus percentage does not necessarily represent conceptual mastery.
- Students may spend more time consuming lectures than practicing questions.
- Revision may be irregular and difficult to schedule.
- Wrong answers are frequently forgotten instead of being converted into a structured mistake-revision cycle.
- Daily study data is rarely converted into meaningful long-term patterns.
- Commercial AI features may require paid APIs, external accounts, internet connectivity, or usage limits.

StudyOS addresses these issues through one integrated, data-driven platform.

3. Project Objectives

- Track study activity at session, day, week, month, subject, topic, and hourly levels.
- Manage academic courses and competitive-examination syllabi.
- Track reading time, pages, resources, and reading progress.
- Create daily, weekly, monthly, course, and exam-oriented goals.
- Track PYQs, question attempts, mock tests, accuracy, and mistakes.
- Schedule revisions and identify overdue revision items.
- Analyze focus, productivity, consistency, time distribution, and weak areas.
- Generate explainable recommendations without paid AI APIs.
- Provide reports that help the user understand preparation patterns.
- Keep the core application usable offline/local-first.

4. Scope of the System

Area	Scope
Academic learning	Courses, subjects, topics, assignments, notes, resources
Competitive exams	Custom exams, syllabus, PYQs, mock tests, performance
Time management	Timer, study sessions, calendar, goals, activity logging
Reading	Books, PDFs, papers, pages, reading duration, progress
Revision	Spaced revision scheduling, flashcards, overdue revision
Analytics	Daily/weekly/monthly dashboards, heatmaps, trends
Personal insights	Study DNA, productive hours, weak areas, time leakage
Data ownership	Local SQLite database, export and backup
AI	No paid AI API required; optional local AI later

5. Core Features

Module	Purpose
Dashboard	Daily study time, goal progress, streaks, current plan, pending revisions, recent sessions.
Study Timer	Start/stop/pause sessions and associate each session with an exam/course/subject/topic.
Courses	Create courses, subjects, topics, subtopics, resources and completion status.
Competitive Exams	Create multiple exams with dates, targets, subjects and custom syllabi.
Reading Tracker	Track books, PDFs, papers, pages, duration and reading progress.
Goals	Daily, weekly, monthly and exam-specific goals.
Calendar	Plan study blocks, revisions, mock tests and tasks.
PYQ/Question Bank	Store questions by exam, year, subject, topic and difficulty.
Mock Tests	Record scores, accuracy, time, section performance and trends.
Revision Engine	Schedule revisions based on topic completion and performance.
Mistake Book	Store incorrect questions, reasons, corrections and reattempt status.
Analytics	Time, consistency, focus, subject balance, accuracy and progress.
Recommendations	Rule-based suggestions for what to study, revise or practice next.
Reports	Daily, weekly, monthly and exam-preparation reports.
Achievements	Milestones such as study hours, PYQs, streaks and completed topics.

6. Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Vite	Interactive single-page application
Language	JavaScript or TypeScript	Frontend implementation
Styling	CSS	Responsive user interface

Layer	Technology	Purpose
Charts	Recharts	Analytics visualization
Icons	Lucide React	Consistent interface icons
HTTP	Axios / Fetch	Frontend-backend communication
Backend	Python 3.10 + Flask	REST API and application logic
ORM	SQLAlchemy	Database models and queries
Database	SQLite	Local, lightweight relational storage
Authentication	JWT / secure session strategy	User authentication
Validation	Marshmallow/Pydantic-style validation	API input validation
Testing	Pytest + frontend test tools	Automated testing
Deployment	Gunicorn + Render/VPS/local	Production hosting

7. System Architecture

```
React + Vite Frontend
|
| HTTP/REST
v
Flask Application
|
+-----+------------------+
| |
Business Logic Analytics Engine
| |
+-----------+-------------+
|
SQLAlchemy
|
SQLite
|
Backup / Export Files
```

The architecture separates presentation, API, business logic, analytics, and persistence. This makes the application easier to test, maintain, and extend.

8. Module Architecture

```
STUDYOS
|
+-- Authentication
+-- Dashboard
+-- Courses
| +-- Subjects
| +-- Topics
| +-- Resources
|
+-- Exams
| +-- Syllabus
| +-- PYQs
| +-- Mock Tests
|
+-- Study
| +-- Timer
| +-- Sessions
```

```
| +-- Reading
| +-- Calendar
|
+-- Revision
| +-- Scheduler
| +-- Flashcards
| +-- Mistakes
|
+-- Analytics
| +-- Time Analytics
| +-- Performance
| +-- Productivity
| +-- Study DNA
|
+-- Reports
+-- Settings
```

9. User Workflow

- Create an account or local profile.
- Add academic courses and/or competitive exams.
- Configure subjects, topics and syllabus.
- Set daily and weekly study targets.
- Start a study session and select what is being studied.
- Stop the session when finished; the system stores duration and context.
- Record practice questions, reading, revision and mock-test activity.
- Review the dashboard daily.
- Use analytics to identify weak topics and time patterns.
- Follow the generated study/revision recommendations.
- Review weekly and monthly reports.

10. Dashboard Design

The dashboard is the main control center. It should prioritize information needed today and avoid excessive visual complexity.

```
Today's Study: 06h 42m
Goal: 08h 00m
Progress: 83.7%
Current Streak: 12 days

Subject Time
OS 02h 20m
DBMS 01h 40m
Python 01h 12m
Math 00h 50m

Today's Priorities
1. OS - Deadlock revision
2. DBMS - Normalization practice
3. GATE PYQ - 30 questions

Alerts
- 2 revisions overdue
- Compiler not studied for 8 days
- Mock test scheduled today
```

11. Course & Subject Management

Courses represent broad learning programs such as B.Tech CSE, Python, Machine Learning, Data Analytics, or a certification. Each course can contain subjects, topics, subtopics and resources.

- Create/edit/archive courses.
- Assign a target completion date.
- Track topic completion.
- Attach notes and resources.
- Record study time against a topic.
- Calculate course-level progress.

12. Competitive Exam Management

The system should not be hard-coded to one examination. Users can create any exam and define its own syllabus.

```
Exam
- Name
- Exam Date
- Target Score
- Target Rank (optional personal field)
- Subjects
- Topics
- Weight/priority
- Revision strategy
- Mock-test history
```

Examples can include GATE CSE, SSC examinations, banking examinations, railway examinations, UPSC-oriented preparation, state-level government examinations, or a custom exam. The software itself does not predict selection outcomes; it reports the user's logged preparation data.

13. Syllabus Tracking

Syllabus should be hierarchical: Exam → Subject → Topic → Subtopic.

```
GATE CSE
|
+-- Operating System
| +-- Processes 100%
| +-- Threads 80%
| +-- Scheduling 70%
| +-- Deadlock 30%
| +-- Memory 0%
|
+-- DBMS
+-- SQL 60%
+-- Normalization 30%
```

Progress should be calculated from topic states rather than entered manually only. Optional weighting can allow important topics to contribute more to the preparation dashboard.

14. Study Session & Timer

Every study session is a structured record.

```
Session ID
User ID
```

```
Start Time
End Time
Duration
Exam ID (optional)
Course ID (optional)
Subject ID
Topic ID
Mode
Focus Rating
Distraction Count
Notes
```

Supported modes can include Learning, Revision, Practice, PYQ, Mock Test, Reading, Coding, Lecture, and Assignment.

15. Reading Tracker

Reading is tracked separately because page progress and reading speed provide information that a generic timer does not.

```
Reading Item
Title
Type: Book/PDF/Paper/Article/Notes
Author
Total Pages
Current Page
Progress %
Session Duration
Pages Read
Reading Speed
Last Read
Next Review
```

The system can show daily pages, monthly reading hours, unfinished resources, and reading consistency.

16. Goals & Task Management

Goal Type	Examples
Daily	8 study hours, 30 PYQs, 20 pages, 1 revision
Weekly	50 study hours, 200 PYQs, 2 mocks
Monthly	3 subjects, 8 mocks, 200 study hours
Exam	Complete syllabus by date, revise all subjects
Course	Finish Python module, complete project chapter

Goals should be measurable and linked to actual activity where possible. For example, study-hour progress can be calculated from study sessions instead of requiring manual updates.

17. Calendar & Planner

The calendar stores planned blocks and actual activity separately. This distinction allows the system to calculate plan-versus-actual performance.

```
Planned:
19:00-20:00 DBMS
20:15-21:45 OS
```

```
22:00-22:30 Revision

Actual:
DBMS = 58 min
OS = 91 min
Revision = 12 min
```

The smart planner can move unfinished tasks to future slots while respecting available study time.

18. PYQ & Question Bank

Questions are categorized using exam, year, subject, topic, difficulty, source and optional marks.

```
Question
- Text
- Exam
- Year
- Subject
- Topic
- Difficulty
- Correct Answer
- Explanation
- Marks
- Negative Marks

Attempt
- Question
- User Answer
- Correct/Incorrect
- Time Taken
- Attempt Date
```

The system can calculate accuracy by subject, topic, year, difficulty and exam.

19. Mock Test Analytics

A mock test record should include total marks, score, attempted questions, correct answers, wrong answers, skipped questions, time taken and section-wise performance.

```
Mock Test #12
Score: 62/100
Accuracy: 74%
Time: 2h 47m

OS 82%
DBMS 76%
CN 71%
TOC 55%
COA 63%
Math 81%
```

The system should show trends over multiple tests and identify topics where incorrect answers repeatedly occur.

20. Revision Engine

The revision engine schedules future reviews when a topic is completed or when a question is answered incorrectly.

```
Topic Learned
|
+--> Revision 1: +1 day
```

```
+--> Revision 2: +3 days
+--> Revision 3: +7 days
+--> Revision 4: +14 days
+--> Revision 5: +30 days
```

The intervals should be configurable. Performance can shorten or extend the next interval. This is a rule-based spaced-revision system and does not require an external AI API.

21. Mistake Book

Every wrong question can be saved into a personal mistake book.

```
Mistake
Question
Subject
Topic
User Answer
Correct Answer
Reason for Mistake
Concept Note
Reviewed?
Reattempted?
Result After Reattempt
```

Possible mistake reasons: concept gap, calculation error, careless error, misunderstood question, memory issue, time pressure, or guessed answer.

22. Notes & Knowledge Library

The library stores personal notes and references. Resources can be tagged by course, exam, subject and topic.

- PDF metadata
- Books
- Research papers
- Web references
- Lecture references
- Personal notes
- Flashcards

The initial system can store metadata and local file paths rather than attempting to become a complete cloud document-management platform.

23. Focus & Distraction Tracking

At the end of a study session, the user can rate focus from 1 to 5 and optionally record distractions.

```
Focus: 4/5
Distractions: 3
Focused Time: 52 min
Interrupted Time: 8 min
Reason: mobile notifications
```

The application can aggregate this data to identify sessions with high focus and time periods with frequent interruptions.

24. Analytics Engine

The analytics engine converts raw activity into interpretable metrics.

Metric	Calculation concept
Daily study time	Sum of completed study-session durations for a date
Average session	Total study duration / number of sessions
Consistency	Days with meaningful study / days in selected period
Goal completion	Completed measurable target / target
Topic accuracy	Correct attempts / total attempts
Reading progress	Current pages / total pages
Plan adherence	Actual completed planned blocks / planned blocks
Focus average	Mean focus rating across sessions
Subject balance	Subject time as a percentage of total study time

25. Smart Recommendation Engine Without Paid AI

The project does not need Gemini, ChatGPT, Claude, OpenAI API, or any paid AI service for its core intelligence. Recommendations can be generated with transparent rules.

```
Recommendation Inputs
- Upcoming exam date
- Topic completion
- Last studied date
- Question accuracy
- Revision due date
- Recent study time
- User availability
- Priority
- Mock-test performance

Rule examples:
IF revision_due <= today
THEN recommend revision.

IF topic_accuracy < 60%
AND attempts >= 10
THEN recommend practice.

IF topic_not_studied >= 7 days
THEN recommend review.

IF exam_date is close
AND syllabus_completion is low
THEN prioritize unfinished high-priority topics.
```

This approach is explainable, deterministic, free to operate, and easy to test.

26. Study DNA & Personal Insights

Study DNA is a long-term summary of patterns discovered from the user's own activity.

```
YOUR STUDY DNA
```

```
Average daily study: 6h 24m
Average session: 57 min
Most productive period: 8 PM-10 PM
Highest-focus subject: Operating System
Least studied subject: Compiler
Average focus: 4.2/5
Weekly consistency: 82%
Average reading: 18 pages/day
```

These are descriptive statistics, not psychological diagnoses or claims about ability.

27. Reports & Export

Reports should be generated for daily, weekly, monthly, course and exam-preparation periods.

- PDF report
- CSV export
- JSON backup
- Study-hour report
- Subject report
- Mock-test report
- Revision report
- Goal report

The user should own the raw data and be able to export it without relying on a paid service.

28. Database Design

Table	Important fields
users	id, name, email, password_hash, created_at
courses	id, user_id, name, description, start_date, target_date
subjects	id, course_id, exam_id, name, priority
topics	id, subject_id, parent_id, name, status, priority
exams	id, user_id, name, exam_date, target_score
study_sessions	id, user_id, subject_id, topic_id, start_time, end_time, duration, mode, focus
reading_items	id, user_id, title, type, total_pages, current_page
reading_sessions	id, reading_item_id, start_time, end_time, pages_read
goals	id, user_id, type, target_value, unit, start_date, end_date
tasks	id, user_id, title, due_date, status, priority
questions	id, exam_id, subject_id, topic_id, year, difficulty, answer
question_attempts	id, question_id, user_id, answer, correct, time_taken
mock_tests	id, exam_id, title, total_marks, duration
mock_attempts	id, mock_id, user_id, score, correct, wrong, skipped
revisions	id, user_id, topic_id, due_date, status, interval_days

Table	Important fields
mistakes	id, user_id, question_id, reason, reviewed, reattempted
notes	id, user_id, subject_id, topic_id, title, content
achievements	id, user_id, type, earned_at
daily_reflections	id, user_id, date, reflection, energy, focus

29. REST API Design

```
Authentication
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout

Dashboard
GET /api/dashboard
GET /api/dashboard/today

Study
POST /api/study-sessions
GET /api/study-sessions
PUT /api/study-sessions/{id}
DELETE /api/study-sessions/{id}

Courses
GET /api/courses
POST /api/courses
GET /api/courses/{id}
PUT /api/courses/{id}

Exams
GET /api/exams
POST /api/exams
GET /api/exams/{id}/syllabus

Analytics
GET /api/analytics/daily
GET /api/analytics/weekly
GET /api/analytics/subjects
GET /api/analytics/heatmap

Questions
GET /api/questions
POST /api/questions
POST /api/questions/{id}/attempt

Mocks
POST /api/mocks
POST /api/mocks/{id}/attempt

Revision
GET /api/revisions/today
POST /api/revisions/{id}/complete
```

30. React Frontend Structure

```
src/
■■■ components/
■ ■■■ Navbar/
■ ■■■ Sidebar/
■ ■■■ StudyTimer/
```

```
■ ■■■ StatCard/
■ ■■■ ProgressBar/
■ ■■■ Heatmap/
■ ■■■ Charts/
■■■ pages/
■ ■■■ Dashboard.jsx
■ ■■■ Courses.jsx
■ ■■■ Exams.jsx
■ ■■■ Syllabus.jsx
■ ■■■ Study.jsx
■ ■■■ Calendar.jsx
■ ■■■ Reading.jsx
■ ■■■ Questions.jsx
■ ■■■ MockTests.jsx
■ ■■■ Revision.jsx
■ ■■■ Mistakes.jsx
■ ■■■ Analytics.jsx
■ ■■■ Settings.jsx
■■■ services/
■ ■■■ api.js
■■■ hooks/
■■■ utils/
■■■ App.jsx
```

31. Flask Backend Structure

```
backend/
■■■ app/
■ ■■■ __init__.py
■ ■■■ config.py
■ ■■■ extensions.py
■ ■■■ models/
■ ■■■ routes/
■ ■■■ schemas/
■ ■■■ services/
■ ■ ■■■ analytics.py
■ ■ ■■■ recommendations.py
■ ■ ■■■ revision.py
■ ■ ■■■ reports.py
■ ■■■ utils/
■■■ migrations/
■■■ tests/
■■■ run.py
■■■ requirements.txt
```

Business rules should be kept in service modules rather than embedded directly inside route functions.

32. Security

- Password hashing with a modern password-hashing library.
- JWT or secure session-based authentication.
- Input validation on every API endpoint.
- Parameterized ORM queries.
- CORS configured for trusted frontend origins.
- Environment variables for secret keys.
- No API keys hard-coded in source code.
- Automatic session timeout where appropriate.
- Exported backups should be protected because they contain personal study history.

If the application is later hosted publicly, HTTPS and secure cookie/token practices are required.

33. Offline-First Strategy

The project should continue to function when internet access is unavailable, especially for the timer, course data, study sessions, notes, and analytics.

```
Option A - Local deployment
React -> Flask -> SQLite

Option B - PWA enhancement
React PWA
|
Local browser storage for temporary state
|
Sync with Flask when available

Option C - Desktop wrapper later
React + local Flask/SQLite
```

The simplest and most reliable first release is local React + Flask + SQLite.

34. Development Roadmap

Phase	Deliverables
Phase 1	Project setup, database, authentication, basic UI
Phase 2	Courses, subjects, topics and syllabus
Phase 3	Study timer and session tracking
Phase 4	Dashboard and basic analytics
Phase 5	Goals, tasks and calendar
Phase 6	Reading tracker and notes
Phase 7	Questions, PYQs and mock tests
Phase 8	Revision scheduler and mistake book
Phase 9	Advanced analytics and Study DNA
Phase 10	Reports, export, testing and deployment

35. Testing Strategy

Test level	Examples
Unit tests	Goal calculations, accuracy, revision intervals, recommendation rules
API tests	Authentication, CRUD endpoints, validation, error responses
Database tests	Relationships, constraints, cascading behavior
Frontend tests	Timer controls, forms, navigation, charts
Integration tests	Create session → database → dashboard update
End-to-end	Login → plan → study → stop timer → analytics → report

Test level	Examples
Security tests	Unauthorized access, invalid tokens, malformed input
Performance tests	Large question banks, many study sessions, analytics queries

36. Deployment

For development, run React and Flask separately. For production, the React application can be built into static assets and Flask can run behind Gunicorn, or the frontend can be hosted separately from the backend.

```
Frontend
npm install
npm run dev

Production build
npm run build

Backend
python -m venv venv
pip install -r requirements.txt
python run.py

Production
gunicorn "app:create_app()"
```

SQLite is appropriate for a personal single-user application. If the system becomes multi-user or heavily concurrent, PostgreSQL is the recommended next step.

37. Future Enhancements

- Optional local language model for note summarization and question generation.
- Browser extension for automatic study-site activity tracking.
- PWA and mobile-friendly interface.
- Desktop application wrapper.
- Calendar synchronization as an optional integration.
- Local speech-to-text for reflections.
- OCR for importing printed notes.
- Advanced statistical forecasting of study-hour requirements, without predicting exam results.
- Custom data-import formats for existing question banks.
- Encrypted local backup.

All future enhancements should remain optional so the core application continues to operate without paid APIs.

38. Project Novelty

The novelty is not a claim that every individual feature is new. Timers, calendars, flashcards, goals, and analytics already exist in various learning products. The project differentiates itself by integrating them around a personal data model that connects time, syllabus, questions, mistakes, revisions, reading, goals, and performance.

Novel combination	Description
Activity-to-topic linkage	Every study session can map to a concrete topic.
Time + performance	Study duration is compared with question/mock performance.
Revision + mistakes	Wrong answers can automatically create revision items.
Plan vs actual	Scheduled study is compared with actual activity.
Study DNA	Long-term descriptive patterns are derived from personal data.
No paid AI dependency	Core intelligence uses deterministic algorithms and statistics.
Local ownership	SQLite/local-first design keeps personal learning data under user control.

39. Suggested Final-Year Project Description

StudyOS is a personal learning and competitive examination tracking platform developed using React.js, Vite, Python Flask, SQLAlchemy, and SQLite. The system enables users to manage academic courses and competitive-examination syllabi, record study sessions, track reading activity, create goals, schedule revisions, practice previous-year questions, analyze mock tests, maintain a mistake book, and monitor long-term productivity. A rule-based analytics and recommendation engine identifies weak topics, overdue revisions, neglected subjects, time-distribution patterns, and recommended next activities. The platform is designed to operate without paid AI APIs such as Gemini or ChatGPT, making its core functionality explainable, locally controllable, and cost-effective. Optional local AI capabilities may be introduced later without changing the core system.

40. Conclusion

StudyOS is designed as a complete personal study-management system rather than a simple timer or to-do application. Its central idea is to transform day-to-day learning activity into structured data and then turn that data into useful, explainable insights.

The first implementation should focus on reliability: accurate study-session tracking, a clean database, correct syllabus progress, dependable goals, and useful analytics. Once those foundations are stable, revision scheduling, question analytics, mistake tracking, Study DNA, and intelligent recommendations can be added.

The system does not require Gemini, ChatGPT, Claude, OpenAI API, or any other paid AI API. A transparent rules-and-statistics engine can provide substantial personalization using only the user's own data. Optional local AI can be introduced later if desired.

Recommended final title: “StudyOS: A Personal Learning, Productivity and Competitive Examination Tracking System Using React.js and Python”.

Appendix A — Recommended Initial requirements.txt

```
Flask==2.3.3
Flask-CORS==4.0.0
Flask-SQLAlchemy==3.0.5
Flask-JWT-Extended==4.5.2
python-dotenv==1.0.1
SQLAlchemy==2.0.x
Werkzeug==2.3.7
Gunicorn==21.2.0
bcrypt==4.0.1
PyJWT==2.8.0
```

Exact package versions can be adjusted after testing the chosen Python 3.10 environment. Avoid installing unnecessary AI/ML packages in the first version.

Appendix B — Recommended npm Packages

```
react
react-dom
react-router-dom
axios
recharts
lucide-react
date-fns
```

Keep the frontend dependency list small. Additional packages should be added only when a concrete feature requires them.

Appendix C — Example Recommendation Rules

Rule	Logic
R1	If a revision is due today or overdue, place it in Today's Priorities.
R2	If topic accuracy < 60% with at least 10 attempts, recommend concept review + practice.
R3	If a topic has not been studied for 7+ days, recommend review.
R4	If a mock section accuracy falls below the user's selected threshold, recommend practice in that topic.
R5	If a high-priority topic is incomplete and exam date is approaching, raise its planning priority.
R6	If planned study repeatedly exceeds available time, reduce daily load and redistribute tasks.
R7	If a subject receives very little study time compared with other subjects, flag it for review.
R8	If a mistake is reattempted incorrectly, shorten its next revision interval.

Appendix D — Minimum Viable Product Checklist

- React + Vite project created
- Flask backend created
- SQLite database connected
- User authentication working
- Course/subject/topic CRUD working

- Study timer working
- Study sessions persisted
- Daily dashboard working
- Daily and weekly goals working
- Basic analytics working
- Responsive UI working
- Backup/export working
- Unit and API tests added

Appendix E — Phase-2 Checklist

- Competitive exam management
- Hierarchical syllabus
- PYQ/question bank
- Question attempts
- Mock tests
- Mock analytics
- Revision engine
- Mistake book
- Reading tracker
- Calendar planner
- Study recommendations